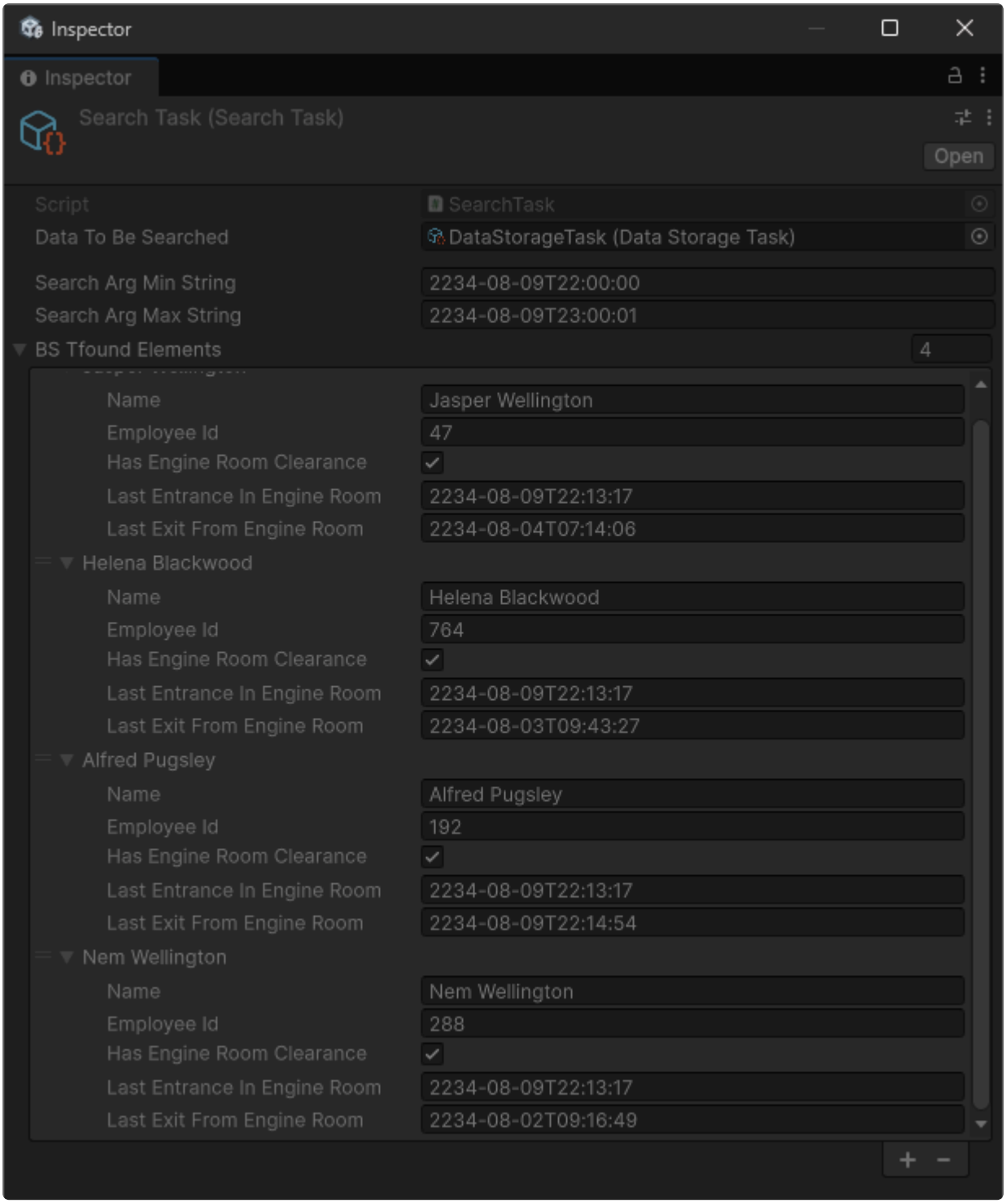
For Week 4, the search/sorting Algorithm I decided on was "Merge Sort" as quick sort was confusing with it's pivot swapping.

For week 5's search algorithm I chose a "Binary Search Tree" (BST), as I didn't wanna do more math then necessary with the hash functions. + I felt like the tree is quite intuitive to work with. (although hard to fully visualise)
For a similar reason I chose to do a Depth First Search (DPS), as exploring all the way down until a leaf is hit then back tracking and checking the other side (left then right, or Right then left).
Felt like the best way to not visit unnecessary nodes.

The suspects I found via my search were as follows:



With this search I then iterated over each element to find the one who also exits within the specified range.

```
for (int i = 0; i < foundElements.Count; i++)
{
    if (DateTime.Parse(foundElements[i].lastExitFromEngineRoom) >=
SearchArgMin && DateTime.Parse(foundElements[i].lastExitFromEngineRoom) <=
SearchArgMax)
    {
        Debug.Log($"Killer Found: {foundElements[i].name}");
        break;
```

```
    }
}
```

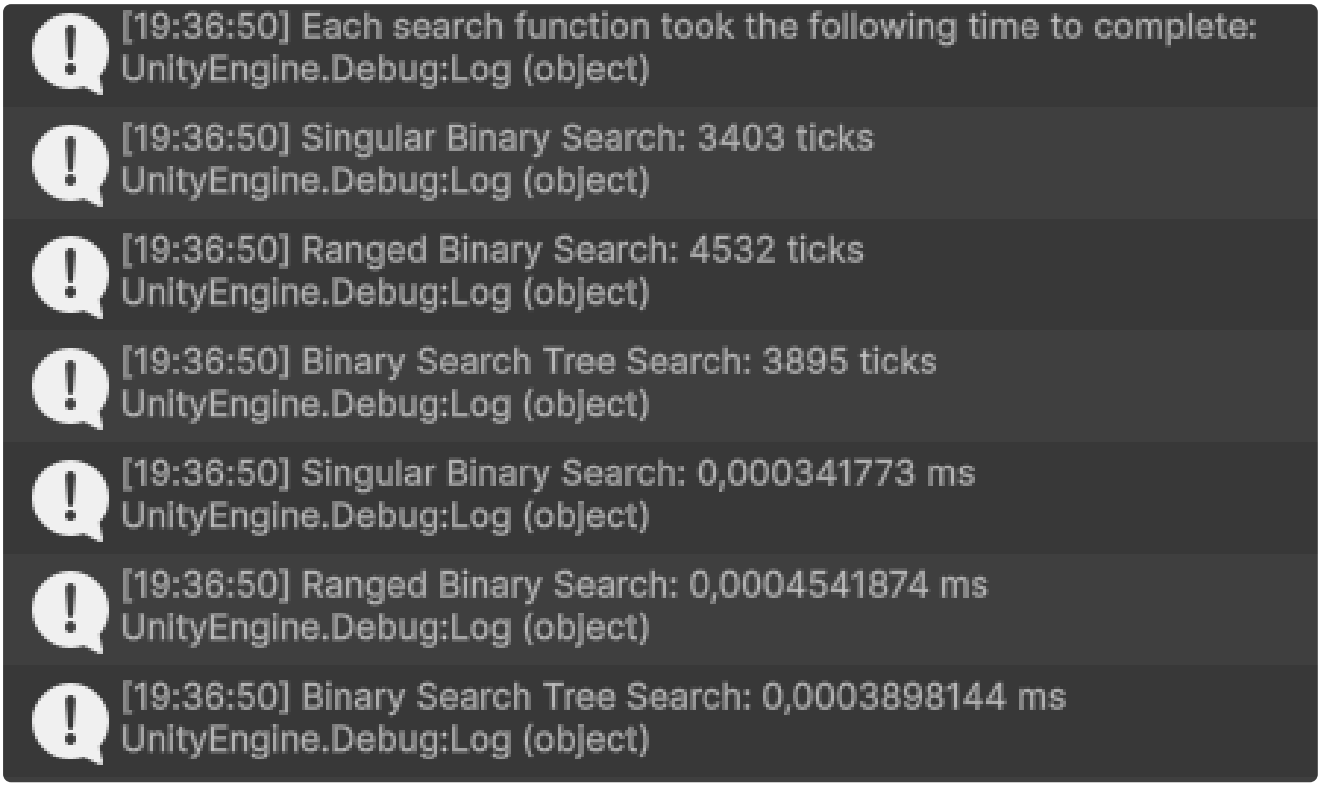
Which results in:



(Don't look at the timeStamp)

Granted I could have found this `Employee` by changing all the times I only check `LastEntranceInEngineRoom` to check `LastExitInEngineRoom` instead and parse that field instead of Entrance. But this was the way I had to do it, In order to hand in everything In time.
(otherwise a generic search *would* have been the best).
(I thought we only ever had to search on *entrance* to find the killer, until I got informed that the assignment instructions had ben updated and stated a specific range + that the exit also needed to be checked.)

The benchmarking is as follows:



See the dev diary for the challenges and how I worked through them.